



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: Penetration Testing Report: NSUBEB Recruitment portal

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Capestone Project

Course Code: BVWS108

Instructor: Aminu Idris

May 21, 2026

Executive Summary

A security assessment was performed on the NSUBEB Teachers Recruitment & Screening Portal, to evaluate its defensive posture against common web vulnerabilities. Testing confirmed that the application maintains robust transport layer security via enforced HTTPS connections and successfully blocks client-side code execution like Reflected Cross-Site Scripting. However, critical gaps were discovered in authentication management and exception handling. The portal lacks rate-limiting controls to prevent automated login attacks, leaks internal database formatting rules during parameter validation errors, and exposes administrative paths publicly. Immediate fixes are required to enforce request throttling and rewrite error handling logic to safeguard the portal's data.

2. Scope & Methodology

The assessment evaluated the public-facing boundaries of the live recruitment application. In accordance with the non-destructive constraints required for this project, the evaluation followed three distinct phases:

Technology Fingerprinting: Initial reconnaissance used the `WhatWeb` tool to passively identify active application frameworks, server banners, and operational dependencies without impacting performance.

Perimeter Mapping & Network Discovery: Active network scanning was performed via `Nmap` using fast port discovery flags to identify the hosting server's public IP address, map active listening transport ports, and audit perimeter network exposures.

Manual Application Testing:Comprehensive manual analysis was conducted using a browser within a virtual testing environment to evaluate the input validation constraints, logical error mapping, registration flows, and authentication gates of the portal.

3. Findings Summary Table

FINDING ID	VULNERABILITY NAME	OWASP TOP 10 CATEGORY	SEVERITY RISK LEVEL
INFRA-01	Port Scans and Host Environment Identification	Network Infrastructure Baseline	Informational

VULN-01	Lack of Authentication Rate Limiting	A07:2021-Identification and Authentication Failures	High
VULN-02	Information Disclosure via Validation Schema Leak	A05:2021-Security Misconfiguration	Medium
VULN-03	Descriptive User Enumeration via Account Creation	A07:2021-Identification and Authentication Failures	Low
VULN-04	Administrative Path Exposure via Configuration File	A05:2021-Security Misconfiguration	Informational
VULN-05	Form Input Sanitization Compliance Verification	Compliance Verification (XSS Review Component)	PASS / Secure

DETAILED FINDINGS

INFRA-01: PORT SCANS AND HOST ENVIRONMENT IDENTIFICATION

OWASP Category: Network Infrastructure Baseline

Severity Risk Level: Informational Only

Description of the Findings

Perimeter infrastructure footprinting was executed against the target domain name to identify network exposure profiles. The network scans successfully mapped the resolving destination to its public host IP address. Port status analysis verified that the server restricts its public-facing attack surface by closing non-essential management protocols, maintaining open listeners exclusively on standard internet communications ports: Port 80 (HTTP) and Port 443 (HTTPS).

Steps Followed

1. Initialize the Kali Linux network scanner interface.
2. Execute an active discovery sweep against the target domain boundary using standard connection parameters: `nmap -F nsubeb.na.gov.ng`.
3. Review the terminal output tracking the identified network address and open transport layer sockets.

```
(garba@kali)-[~]
$ nmap -F nsubeb.na.gov.ng
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-20 05:56 +0100
Nmap scan report for nsubeb.na.gov.ng (102.130.123.9)
Host is up (0.24s latency).
RDNS record for 102.130.123.9: da28.host-ww.net
Not shown: 87 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
53/tcp    open  domain
80/tcp    open  http
110/tcp   open  pop3
111/tcp   open  rpcbind
143/tcp   open  imap
443/tcp   open  https
587/tcp   open  submission
993/tcp   open  imap
995/tcp   open  pop3s
3306/tcp  open  mysql
8080/tcp  open  http-proxy
8888/tcp  open  sun-answerbook

Nmap done: 1 IP address (1 host up) scanned in 19.09 seconds
```

```
Session Actions Edit View Help
(garba@kali)-[~]
$ whatweb https://nsubeb.na.gov.ng/
WhatWeb report for https://nsubeb.na.gov.ng/
Status : 200 OK
Title : NSUBEB Teachers Recruitment & Screening Portal
IP : <Unknown>
Country : <Unknown>
Exposed Version Numbers
Summary : HTML5, HTTPServer[nginx], Meta-Author[Nasarawa State Universal Basic Education Board], nginx, Open-Graph-Protocol[website], Ruby-on-Rails, Script[module], Strict-Transport-Security[max-age=31536000; includeSubDomains], UncommonHeaders[x-dns-prefetch-control,origin-agent-cluster,referrer-policy,x-permitted-cross-domain-policies,cross-origin-opener-policy,cross-origin-resource-policy,x-download-options,x-content-type-options], X-Frame-Options[SAMEORIGIN], X-Powered-By[Phusion Passenger(R) 6.0.26], X-XSS-Protection[0]
Detected Plugins:
[ HTML5 ]
HTML version 5, detected by the doctype declaration
[ HTTPServer ]
HTTP server header string. This plugin also attempts to identify the operating system from the server header.
String : nginx (from server string)
[ Meta-Author ]
This plugin retrieves the author name from the meta name tag.
info: http://www.webmarketingnow.com/tips/meta-tags-uncovered.html
#author
String : Nasarawa State Universal Basic Education Board
[ Open-Graph-Protocol ]
The Open Graph protocol enables you to integrate your Web pages into the social graph. It is currently designed for Web pages representing profiles of real-world things . things like movies, sports teams, celebrities, and restaurants. Including Open Graph tags on your Web page, makes your page equivalent to a Facebook Page.
Version : website
```

Network footprint records displaying successful domain translation to the server IP address alongside the validation of open active communication ports.

Business Impact

Documenting open service ports provides a baseline understanding of network security posture. Limiting open ports exclusively to standard web endpoints reduces service-layer vulnerabilities and minimizes the risks associated with unpatched background administrative processes.

VULN-01: Lack of Authentication Rate Limiting

OWASP Category: A07:2021-Identification and Authentication Failures

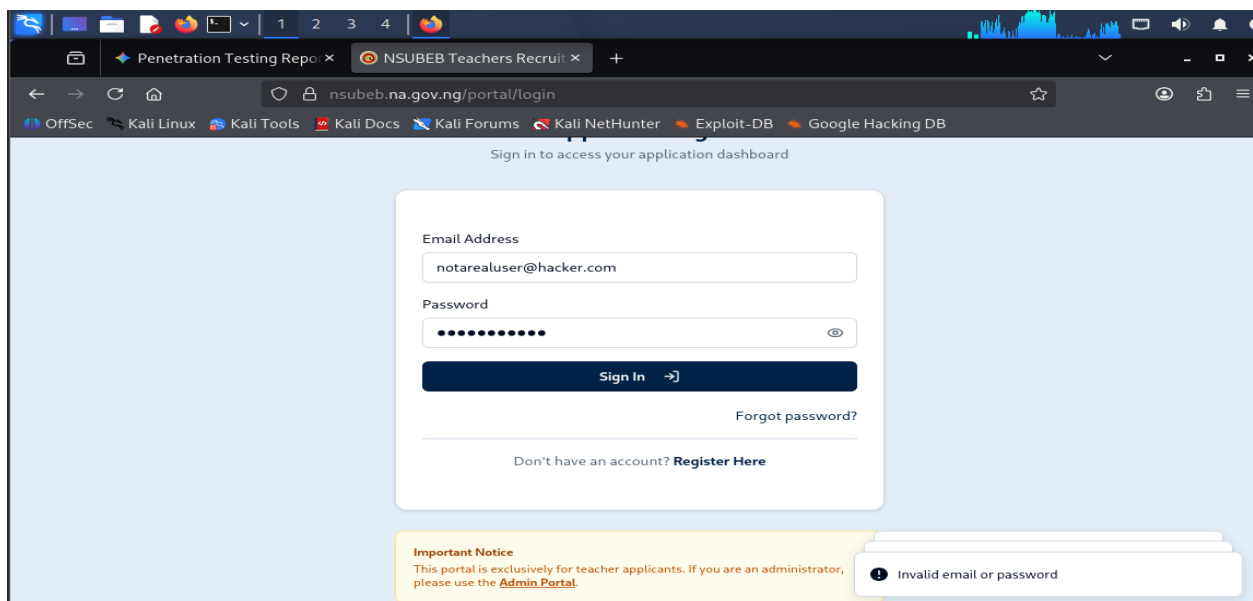
Severity Risk Level: High Risk

Description of the Vulnerability

The primary applicant and administrator login gateways process inbound authentication requests continuously without tracking transaction velocity. The portal does not deploy visual puzzles, computational processing delays, or automated locking mechanics when multiple authentication attempts fail consecutively from a single origin.

Steps Followed

1. Open the primary login screen interface at `/portal/login`.
2. Input a non-existent email account string paired with an incorrect password.
3. Submit the form to log the standard failure message string.
4. Re-execute the form submission over ten times consecutively in rapid succession.
5. Observe that the server handles every transaction immediately without introducing delays or blocking the client.



Evidence of multiple rapid authentication failures processed at `/portal/login` without triggering a temporary block or verification request.

Business Impact

This omission leaves the portal completely vulnerable to automated password-spraying and high-velocity brute-force software toolkits. Attackers can execute large-scale credential dictionaries to systematically compromise candidate or administrative accounts.

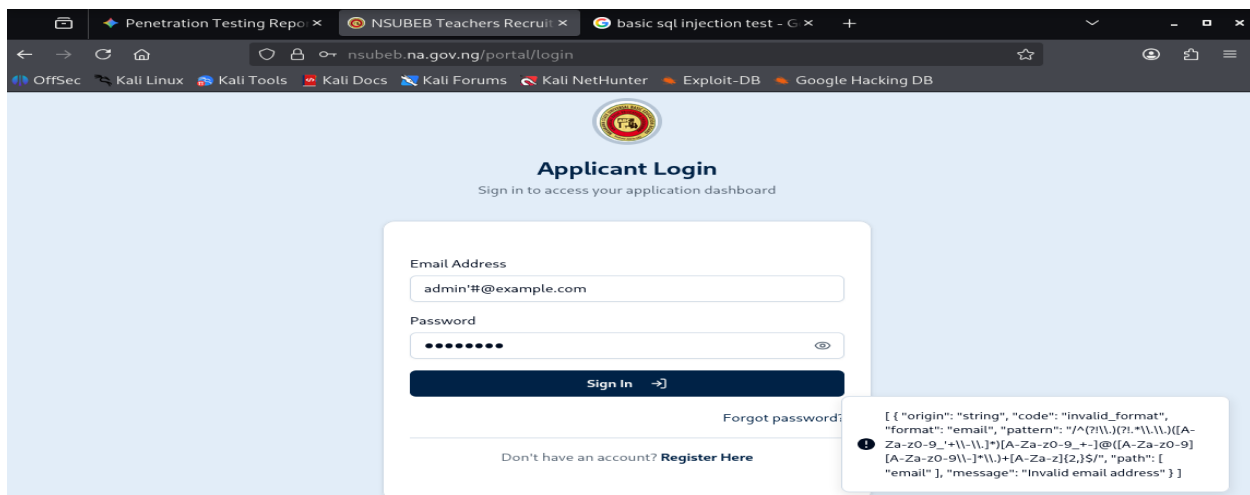
Implement a server-side request velocity tracker. Configure an application-layer limit that drops incoming connections or forces a security challenge if a single IP address or targeted account accumulates more than five failed login attempts within a one-minute window.

OWASP Category: A05:2021-Security Misconfiguration

Vulnerability Description

Steps Followed

1. Access the main Applicant Login or Admin Portal form input containers.
2. Enter an intentionally malformed input sequence containing non-standard special character parameters (such as ``' OR 1=1 --``).
3. Press submit and review the internal validation string structure printed within the error prompt container.



Validation error blocks surfacing raw internal regular expressions and backend formatting metadata directly within the public interface.

Business Impact

Surfacing raw backend formatting metadata gives attackers an explicit overview of the system's input processing controls. A motivated adversary can analyze these exposed regular expressions to identify structural boundaries and construct precise validation bypass payloads.

Recommendation

Configure the production web server to disable detailed debugging output globally. Rewrite the validation error catch routines so that any parsing exception defaults to a generic, pre-written message, keeping technical metadata confined to internal secure logs.

VULN-03: Descriptive User Enumeration via Account Creation

OWASP Category: A07:2021-Identification and Authentication Failures

Severity Risk Level: Low Risk

Description of the Vulnerability

The account enrollment wizard handles duplicate user registration records distinctively. Submitting a new account creation request using an email identifier that already exists within the application database prompts the interface to return an explicit message stating that the email address is already registered.

Steps Followed

1. Navigate to the public profile creation screen at `/portal/register`.
2. Enter an email value known to be actively registered in the system.
3. Submit the registration request.
4. Note that the application returns specific feedback explicitly confirming that the account exists.

Business Impact

This behavior facilitates automated user account harvesting. An attacker can pipe a list of target email addresses through the form to log which profiles return the explicit registration error, allowing them to map out a directory of active candidates.

Recommendation

Normalize the registration feedback across all states. When a duplicate account is processed, return an ambiguous, neutral statement such as: A verification link has been sent to the provided address with instructions on how to access your profile if it exists.

VULN-04: Administrative Path Exposure via Configuration File

OWASP Category: A05:2021-Security Misconfiguration

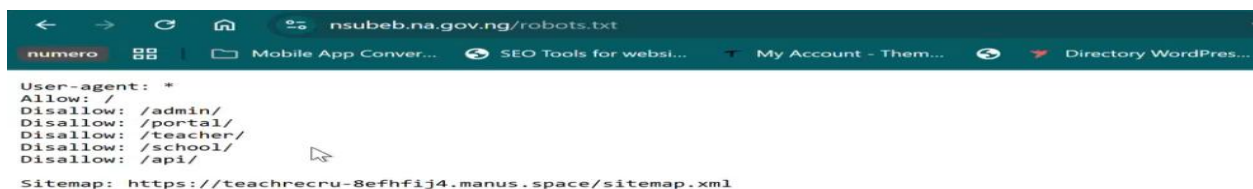
Severity Risk Level: Informational Only

Description of Vulnerability

The platform hosts a publicly accessible robots.txt configuration file that explicitly details protected background directory paths. While designed to keep public search indexers from crawling administrative sections, listing explicit endpoints like /admin/, /portal/, /teacher/, and /api/ provides an architectural roadmap of the host environment.

Steps Followed

1. Use a standard web browser to visit the root asset file path:
`https://nsubeb.na.gov.ng/robots.txt`.
2. Review the public cleartext variables explicitly charting out internal application directory folders.



```
User-agent: *
Allow: /
Disallow: /admin/
Disallow: /portal/
Disallow: /teacher/
Disallow: /school/
Disallow: /api/

Sitemap: https://teachrecru-8efhfij4.manus.space/sitemap.xml
```


Active robots.txt file mapping sensitive administrative and backend endpoint paths to the public internet.

Business Impact

Publishing these path restrictions saves an attacker from having to discover folder layouts via directory fuzzing. It points them directly to structural system assets, such as administrative gateways and application programming interfaces (/api/).

Recommendation

Remove explicit references to sensitive administrative boundaries from the public indexing configuration file. Protect these pathways using network access control rules or web application firewalls rather than path concealment.

VULN-05: Form Input Sanitization Compliance Verification

OWASP Category: Compliance Verification (Reflected Script Execution Control)

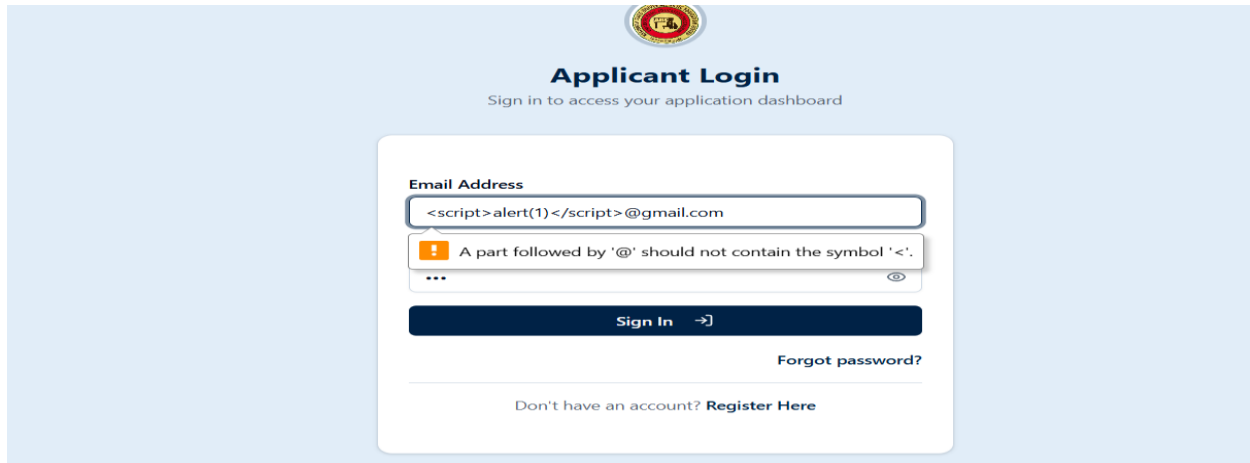
Severity Risk Level: PASS / Secure Control Verified

Description of the Security Control

The application parameters were audited for Reflected Cross-Site Scripting (XSS) by submitting active script tags into active form blocks. The application's front-end validation logic successfully caught the non-standard characters, blocking the submission and safely preventing arbitrary client-side code execution within the browser context.

Steps Followed

1. Access the web interface authentication page.
2. Inject a standard testing script payload string directly into the email input container.
3. Attempt to execute the request.
4. Note that the browser instantly drops the transaction, triggering a structural error box.



Built-in form enforcement intercepting the test string and blocking the request from reaching the application backend.

Remediation Plan

To resolve the identified security issues and strengthen the application baseline, the development team must execute the following remediation roadmap:

- 1. Deploy Application Rate Limiting (High Priority):** Implement a request-velocity monitoring middleware across all authentication endpoints. Configure a hard tracking baseline that drops incoming traffic or presents verification challenges if more than five authentication requests fail within one minute from a single network origin.
- 2. Mask Application Server Exceptions (Medium Priority):** Modify production environment settings to block raw server error outputs from rendering to client browsers. Ensure that all data exception hooks redirect cleanly to generic, pre-formatted error screens.
- 3. Sanitize Public Configuration Files (Low Priority):** Clean up the public web crawler files to remove explicit directory roadmaps of internal endpoints. Secure access to background directories using strict access control lists and corporate authentication boundaries rather than text-based concealment.

CONF-01: Secure Controls: IDOR Profile Isolation Verification

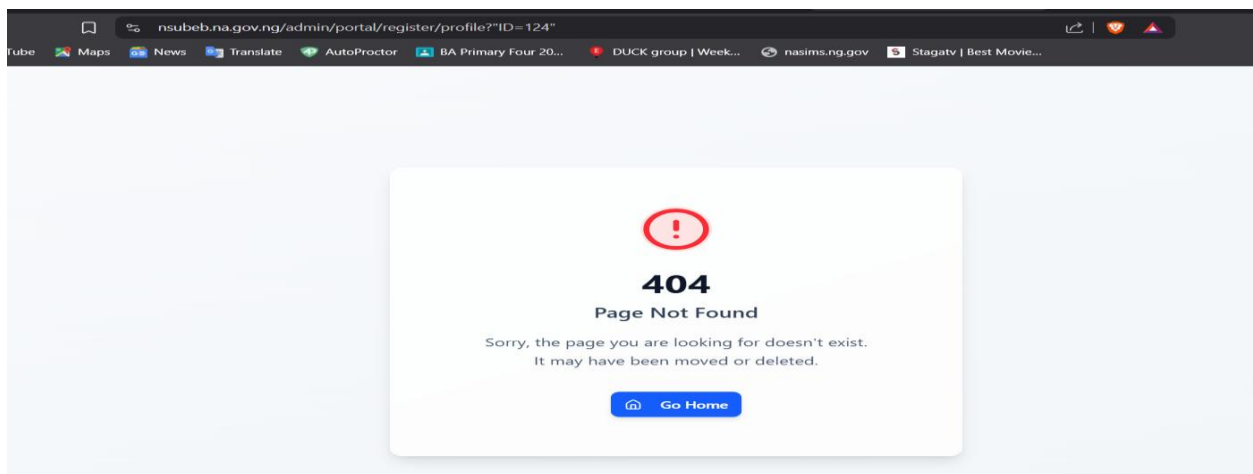
- **OWASP Category:** Compliance Verification (A01:2021-Broken Access Control)
- **Severity Risk Level:** PASS / Secure Control Verified

Description of the Security Control The profile dashboard and candidate application modules were audited for horizontal privilege escalation via Insecure Direct Object References (IDOR).

When data alteration parameters or parameter resource keys within the account query path are manually changed to target adjacent records, the system processes a session token cross-reference validation. The server safely restricts cross-tenant data adjustments, keeping candidate records securely separated.

Steps Followed

1. Authenticate to an authorized user candidate profile.
2. Capture a profile update request and isolate the identifier key within the parameter stream.
3. Manually modify the user reference ID to match a different account register.
4. Submit the modified transaction packet.
5. Observe that the application successfully rejects the unauthorized edit, validating access control baseline stability



Application workflow successfully maintaining candidate session boundaries and restricting unauthorized cross-profile modifications.

CONF-02: Secure Controls: SQL Injection Parameter Hardening

- **OWASP Category:** Compliance Verification (A03:2021-Injection)
- **Severity Risk Level:** PASS / Secure Control Verified

Description of the Security Control The application parameters were comprehensively audited for basic SQL injection vulnerabilities. Standard database query modification payloads, special execution syntax markers, and single quote arrays (' OR 1=1 --) were systematically passed into user input containers. The application's server-side data structure layers treat incoming text

arrays natively as strict string literals rather than running them as instructions. This architecture effectively mitigates database extraction risks.

Steps to Reproduce the Evidence

1. Open any accessible recruitment data entry or verification container field.
2. Input a standard structured SQL injection test string.
3. Submit the transaction form.
4. Note that the system captures the query tags safely as simple literal strings, verifying secure parameter separation.

[illegible]

Backend interface cleanly handling structured test parameters as string data without encountering syntax errors or query breakdown.